

IP-SAKTI Sahayak — Economic Feasibility & Cost Optimization

Purpose: A concise 2-page extraction of the most important points from the full 7-page planning document, with each point explained in simple language and its direct use in IP-SAKTI Sahayak.

1. Economic Feasibility Must Be Built into the Architecture

- **Meaning:** Cost should be considered from the beginning, not after deployment.
- **Key idea:** Control LLM calls, tokens, latency and infrastructure costs; use local/open-source processing where practical.
- **Cost includes:** LLM + embeddings + database + server + storage + bandwidth + monitoring + external APIs + maintenance.
- **Use in our project:** Makes the RAG chatbot economically scalable instead of paying for every pipeline step on every query.

2. Reduce Unnecessary LLM Calls

- **Gemini's role:** Use it mainly for final answer generation, not as the default processor for every task.
- **Abstention:** If retrieved evidence is below the confidence threshold, do not generate an unsupported answer; return a safe escalation.
- **Deterministic tasks:** Classification, validation and jurisdiction routing can run without an LLM.
- **Cache/FAQ:** Stable, frequently asked questions can be answered without a fresh model call.
- **Use in our project:** Directly lowers API cost and reduces hallucination risk because Gemini is called only when useful evidence exists.

3. Control Token Consumption

- **Do not send the whole corpus:** Retrieval must first narrow the evidence.
- **Use:** Controlled top-k retrieval, cross-encoder reranking, removal of duplicate/overlapping chunks, context/output limits and concise responses.
- **Illustrative starting budget:** ~500 system + ≤500 user + ≤1,500 retrieved evidence + ≤500 generated answer = ~3,000 tokens.
- **Important:** These are starting values and must be calibrated using real query/answer-quality measurements.
- **Use in our project:** Keeps each grounded answer cheaper and faster while still giving Gemini the evidence it needs.

4. Layered & Knowledge-Versioned Caching

- **Exact-query cache:** Reuse answers for normalized repeated questions, including citations, confidence, jurisdiction and knowledge-base version.
- **Semantic cache:** Reuse differently worded questions only when intent, jurisdiction, language, source validity and knowledge version are compatible.
- **Source-driven invalidation:** If a regulation/source changes, invalidate only dependent answers.
- **Use in our project:** Especially important for regulatory/IP answers because old cached information must not survive a source update.

5. Local / Open-Source Processing

- **Local components:** BGE-M3 embeddings, BM25 retrieval, PII scrubbing, classification and confidence/guardrail checks where practical.
- **Important:** Open-source does not mean zero cost; self-hosted components still consume CPU, RAM, storage and operational resources.
- **Use in our project:** Reduces repeated external API expenditure and lets cheaper local layers handle work before Gemini.

6. Metadata Filtering + Smart Retrieval

- **Filter before retrieval:** Use jurisdiction, domain, source, section, language, authority and effective dates.
- **Example:** India + patent + Patents Act 1970 + section 3(p) + relevant authority.
- **Adaptive retrieval:** Start cheaper; if evidence is already strong, potentially skip expensive reranking. Otherwise use BM25/vector combination and cross-encoder.
- **Benchmark:** Compare vector-only, vector+BM25 and vector+BM25+reranker using retrieval quality, CPU/RAM and P50/P95 latency.
- **Use in our project:** Improves relevance and avoids spending computation on irrelevant documents, especially as the AYUSH/IP corpus grows.

7. Parallel Processing + Prompt/Model Caching

- **Parallel retrieval:** Vector and BM25 retrieval can run concurrently and then be combined with Reciprocal Rank Fusion.
- **Stable prompts:** Keep fixed system/safety instructions in a consistent prefix and put dynamic query/evidence later.
- **Provider abstraction:** Keep the model layer replaceable so the project is not locked to one provider.
- **Use in our project:** Reduces wall-clock latency and preserves the option to move between hosted or local models later.

8. Rate Limits, Quotas & Cost Controller

- **Protection:** Limit per-IP/account requests, bursts, concurrency, question length and context size.
- **Budgets:** Maintain per-user and system-wide daily/monthly usage limits.
- **Cost-control flow:** cache check → confidence check → budget check → token estimation → compression if needed → LLM call → usage record.
- **Use in our project:** Prevents accidental or automated excessive Gemini usage and supports a controlled free/paid model later if required.

9. Fallback + Infrastructure Scaling

- **Fallback:** If the LLM provider is unavailable, use cached answers and authoritative retrieved evidence in a controlled response, clearly stating generation is unavailable.
- **Scale gradually:** Prototype can use FastAPI + ChromaDB + SQLite + local embedding + hosted LLM.
- **Later:** Multiple API instances, shared cache, stronger DB and load balancing; at large scale, horizontally scaled API and scalable storage.
- **Do not over-engineer early:** Kubernetes, GPU clusters and complex microservices should be introduced only when measurements justify them.
- **Use in our project:** Keeps the prototype inexpensive while providing a clear path to support more users.

10. Storage & Audit Logging

- **Separate responsibilities:** Vector DB for embeddings/retrieval metadata; relational DB for users, source versions, audits and cache metadata; object storage for PDFs/backups/large artifacts.
- **Compact logs:** Store request ID, query hash/scrubbed query, jurisdiction, source IDs, confidence, model, token usage, cache hit/miss, latency and timestamp.
- **Use sampled traces:** Do not store huge prompts/contexts for every request unless necessary.
- **Use in our project:** Gives traceability for evidence-grounded answers without creating unnecessary storage cost.

11. Monitor Cost AND Answer Quality Together

- **Economic metrics:** cost/query, tokens/query, cache-hit rate, LLM-call rate.
- **Performance:** P50 and P95/P99 latency, API failure rate.
- **RAG quality/safety:** abstention rate, retrieval precision and citation accuracy.
- **Use in our project:** The goal is not merely the cheapest response; it is the lowest cost per reliable, grounded answer.

12. Calibrate Similarity Threshold + Recommended Architecture

- **Threshold:** The current cosine-distance guardrail is only a starting point. Test representative AYUSH/IP questions with known relevant sources.
- **Measure:** retrieval quality, false acceptance, false abstention, latency and cost at different thresholds.
- **Architecture:** API Gateway/Rate Limiter → PII Scrubber → Cache → Query Normalizer → Metadata Filter → Local Embedding + BM25 → Hybrid Retrieval → Confidence Check → Conditional Reranker → Evidence Filter → Token Budget/Compression → Gemini → Response Validation → Cache.
- **Use in our project:** This makes Gemini the expensive final layer, reached only after cheaper layers establish that generation is useful and sufficiently supported.

13. Implementation Priority

- **1:** Instrument token usage and latency at every pipeline stage.
- **2:** Implement exact-query caching; then knowledge-versioned cache metadata and source-driven invalidation.
- **3:** Benchmark vector/BM25/reranker combinations and calibrate the similarity threshold.
- **4:** Add strict input/output token budgets and semantic caching only after correctness safeguards.
- **5:** Add rate limits and budget monitoring before public scaling; then provider abstraction and fallback.
- **6:** Move from SQLite/ChromaDB to scalable shared infrastructure only when traffic requires it.
- **Use in our project:** Gives the team a practical order of implementation instead of trying to build every optimization at once.

Core takeaway: IP-SAKTI should optimize for the *lowest cost per reliable, grounded answer*, while maintaining acceptable latency and safety. The document explicitly says cost figures and thresholds must be validated against actual provider pricing, traffic, hardware and evaluation results before production.